

Two-echelon inventory routing on real e-commerce orders from São Paulo state: a reproducible benchmark and a comparison of exact and heuristic methods

Mohammad Hajibabaie
TU Clausthal

Abstract

This report studies a two-echelon inventory routing problem (2E-IRP) built from public order data of the Brazilian marketplace Olist: trucks bring goods from a central warehouse to regional hubs, vans deliver from the hubs to demand points over several days, and no point may run out of stock. We derive 19 instances from 40,501 delivered orders in São Paulo state, with road distances from OSRM and costs taken from Brazilian freight tables. One independent checker scores four methods: a mixed-integer program solved by Gurobi, a greedy baseline, an adaptive large neighborhood search (ALNS), and a matheuristic that adds a linear program for the quantities to the ALNS. Gurobi proves optimality on all 12 small instances, and both the ALNS and the matheuristic reach the optimum in every one of 10 runs per instance. On the medium and large instances both heuristics beat the greedy baseline in every run (Wilcoxon signed-rank test, $p = 0.0020$ on each instance), while the difference between the ALNS and the matheuristic is not significant on any instance. The linear program is called only when the just-in-time quantity rule overloads a van, and in our runs this happened on the large instances only.

1 Introduction

In an inventory routing problem (IRP) one supplier decides, for every period of a horizon, which customers to visit, in which order, and how much to deliver, so that no customer runs out of stock and the sum of routing and holding costs is as small as possible (Archetti et al., 2007; Coelho et al., 2014). The two-echelon version adds a middle layer: large vehicles bring goods from a central warehouse to intermediate depots, and smaller vehicles serve the customers from there. The survey of Cuda et al. (2015) names city logistics as the main motivating example of two-echelon distribution. In that setting, large trucks stay outside the dense urban area and small vehicles serve it.

E-commerce last-mile delivery has this structure. Orders arrive every day at many small addresses, a long-haul leg connects the sellers to the metropolitan areas, and short van routes serve the local pickup points. When the carrier also controls the stock at the pickup points, it can trade fewer and fuller deliveries against the cost of the goods waiting in stock. This trade-off is the one the IRP models.

The 2E-IRP instances of Charaf et al. (2024a) are generated from the single-echelon IRP benchmark of Archetti et al. (2007) by placing satellites and suppliers on rings around the customer area. This project builds a small benchmark from real order data instead. It contributes a reproducible pipeline from the public Olist dataset (Olist, 2021) to 19 frozen 2E-IRP instances with road distances and sourced costs, and a comparison of an exact mixed-integer program (MIP), a greedy rule, an ALNS and an ALNS-based matheuristic on them, with every solution verified by one independent checker.

2 Related work

Inventory routing. Archetti et al. (2007) give a branch-and-cut algorithm for the single-vehicle

IRP and compare the order-up-to policy with the maximum-level (ML) policy, under which any quantity up to the customer capacity may be delivered. Coelho et al. (2014) survey and classify IRP variants. Exact methods for multi-vehicle IRPs include the branch-and-cut of Coelho and Laporte (2013) and the formulations of Adulyasak et al. (2014), who find that formulations with a vehicle index and symmetry breaking are better at finding optimal solutions. Solyali and Süral (2011) propose a strong formulation of the replenishment part under the order-up-to policy.

Two-echelon inventory routing. Guimarães et al. (2019) study a two-echelon multi-depot IRP and solve it with a matheuristic in which a metaheuristic handles the vehicle routes and a subproblem is solved exactly. Farias et al. (2021) formulate a 2E-IRP with routing at both levels, solve it by branch-and-cut with valid inequalities, and add a two-step matheuristic. Rohmer et al. (2019) combine an ALNS with a reduced formulation for a two-echelon IRP of perishable products. Charaf et al. (2024a) solve the 2E-IRP by branch-and-price, and Charaf et al. (2024b) combine tabu search with mathematical programming models. We take the order of operations within one period from Charaf et al. (2024a).

Large neighborhood search and matheuristics. Large neighborhood search removes a part of a solution and inserts it again (Shaw, 1998). Ropke and Pisinger (2006) add adaptive weights that select among several removal and insertion heuristics, and Pisinger and Ropke (2007) apply the same framework to several vehicle routing problems. Voigt (2025) ranks ALNS operators in a meta-analysis of published studies; sequence-based removal ranks best among the removal operators and regret insertion among the insertion operators. Coelho et al. (2012a) solve a multi-vehicle IRP by an ALNS in which some subproblems are solved exactly. Coelho et al. (2012b) solve a single-vehicle IRP with transshipment by an ALNS; once the routes are fixed, a minimum cost network flow sets all quantities and stocks. Archetti et al. (2012) and Archetti et al. (2017) combine tabu search with MIP models, and Solyali and Süral (2022) solve a sequence of MILPs built from the problem formulation. Our matheuristic follows the route-then-quantity split of Coelho et al. (2012b).

3 Problem and MIP formulation

A central warehouse (node 0) supplies k hubs. Each demand point belongs to one fixed hub, its nearest one. In each day of the horizon the following happens in this order (Charaf et al., 2024a): trucks bring goods from the warehouse to hubs, vans load at the hubs and deliver to the demand points, the points consume the demand of the day, and holding cost is charged on the stock left at the end of the day. Goods that reach a hub in the morning may leave on a van on the same day. The warehouse has unlimited stock and no holding cost. Stocks follow the ML policy.

The first echelon uses direct trips: a truck drives from the warehouse to one hub and back, at most once per hub and day. Trucks are hired per trip, so the fleet is not limited. A trip to hub h costs $g_h = f^1 + 2c^1\delta_{0h}$, a fixed charge plus the per-km rate on both legs. Each hub has m vans. A van drives at most one route per day, and each point is visited at most once per day, by one van.

Sets and parameters. H is the set of hubs, N_h the points of hub h , $N = \bigcup_h N_h$, and $\mathcal{T} = \{0, \dots, T-1\}$ the days. $B = \{1, \dots, m\}$ indexes the vans of each hub. $V_h = \{h\} \cup N_h$ and $A_h = \{(a, c) \in V_h \times V_h : a \neq c\}$ are the nodes and arcs of hub h . d_{it} is the demand of point i on day t , U_i and C_h are the capacities of point i and hub h , and I_i^0 and I_h^0 are the initial stocks. Q^1 and Q^2 are the truck and van capacities, c^1 and f^1 the truck cost per km and per trip, c^2 the van cost per km, δ_{ac} the road distance in km, and η^H and η^P the holding costs per kg and day at hubs and points.

Variables. $z_{ht} \in \{0, 1\}$ is 1 if a truck drives to hub h on day t , and $y_{ht} \geq 0$ is the quantity it brings. For van b of hub h on day t : $x_{acbt} \in \{0, 1\}$ is 1 if the van drives arc $(a, c) \in A_h$, $v_{ibt} \in \{0, 1\}$ is 1 if it visits point $i \in N_h$, $w_{hbt} \in \{0, 1\}$ is 1 if it is used, $q_{ibt} \geq 0$ is the quantity it delivers to point i , and $u_{ibt} \in [1, |N_h|]$ is the position of point i on its route. $I_{ht} \geq 0$ and $I_{it} \geq 0$

are the end-of-day stocks, with $I_{h,-1} = I_h^0$ and $I_{i,-1} = I_i^0$.

Model. In the index sets below, $h \in H$, $i, j \in N_h$, $b \in B$ and $t \in \mathcal{T}$.

$$\begin{aligned} \min \quad & \sum_{t \in \mathcal{T}} \sum_{h \in H} g_h z_{ht} + c^2 \sum_{h \in H} \sum_{b \in B} \sum_{t \in \mathcal{T}} \sum_{(a,c) \in A_h} \delta_{ac} x_{acbt} \\ & + \sum_{t \in \mathcal{T}} \left(\eta^H \sum_{h \in H} I_{ht} + \eta^P \sum_{i \in N} I_{it} \right) \end{aligned} \quad (1)$$

$$\text{s.t. } y_{ht} \leq \min(C_h, Q^1) z_{ht} \quad \forall h, t \quad (2)$$

$$I_{h,t-1} + y_{ht} \leq C_h \quad \forall h, t \quad (3)$$

$$I_{ht} = I_{h,t-1} + y_{ht} - \sum_{i \in N_h} \sum_{b \in B} q_{ibt} \quad \forall h, t \quad (4)$$

$$I_{i,t-1} + \sum_{b \in B} q_{ibt} \leq U_i \quad \forall h, i, t \quad (5)$$

$$I_{it} = I_{i,t-1} + \sum_{b \in B} q_{ibt} - d_{it} \quad \forall h, i, t \quad (6)$$

$$\sum_{b \in B} v_{ibt} \leq 1 \quad \forall h, i, t \quad (7)$$

$$\sum_{c \in V_h \setminus \{i\}} x_{icbt} = v_{ibt}, \quad \sum_{c \in V_h \setminus \{i\}} x_{cibt} = v_{ibt} \quad \forall h, i, b, t \quad (8)$$

$$\sum_{j \in N_h} x_{hjb} = w_{hbt}, \quad \sum_{j \in N_h} x_{jhbt} = w_{hbt} \quad \forall h, b, t \quad (9)$$

$$q_{ibt} \leq \min(U_i, Q^2) v_{ibt} \quad \forall h, i, b, t \quad (10)$$

$$\sum_{i \in N_h} q_{ibt} \leq Q^2 w_{hbt} \quad \forall h, b, t \quad (11)$$

$$u_{jbt} \geq u_{ibt} + 1 - |N_h|(1 - x_{ijbt}) \quad \forall h, i \neq j, b, t \quad (12)$$

$$w_{hbt} \leq w_{h,b-1,t} \quad \forall h, t, b \geq 2 \quad (13)$$

The objective (1) adds the truck trip, van routing and holding costs. Constraint (2) allows a shipment to a hub only on a trip day. Constraint (3) is the ML policy at a hub: the stock before van loading plus the truck delivery fits in the hub. Constraint (4) is the hub stock balance; with $I_{ht} \geq 0$, vans load only what the hub holds. Constraint (5) is the ML policy at a point. Constraint (6) is the point stock balance, and $I_{it} \geq 0$ is the no-stockout rule. Constraint (7) lets at most one van visit a point on a day. Constraint (8) gives a visited point one arc in and one arc out, both of the same van. Constraint (9) makes a used van leave its hub once and return once. Constraint (10) allows a delivery only on a visit. Constraint (11) is the van capacity and, since only m vans exist, also the fleet limit. Constraint (12) is the subtour elimination of Miller et al. (1960): an arc between two points puts the second one later on the route, so every cycle passes through the hub. Constraint (13) breaks the symmetry of identical vans: van b runs only if van $b - 1$ runs.

The model is implemented in `gurobipy` with `MIPGap 0`, one thread, and `IntegralityFocus 1`. The last setting matters for correctness: Gurobi accepts a binary variable up to 10^{-5} away from an integer, and through (10) a value of v_{ibt} read as 0 could otherwise carry a small positive delivery that the checker rejects. Binaries are rounded when the solution is read back, and the objective value is compared with the cost the checker computes (Section 5.4).

4 Data and instances

Orders. The Olist dataset (Olist, 2021) contains orders of a Brazilian marketplace from 2016 to 2018. We keep orders of customers in São Paulo state (SP) with status *delivered*: 40,501 orders that have at least one item. The weight of an order is the sum of the product weights of

its items, and its date is the purchase date. We drop 4 orders with a missing item weight. Each zip code prefix (five digits) is placed at the median latitude and longitude of its geolocation rows inside an SP bounding box. 13 prefixes without a coordinate (14 orders) are dropped, which leaves 5,545 prefixes.

Demand folding. A single real week is too thin for an IRP: most prefixes order nothing on most days. We therefore fold one year of orders onto one week. The demand of prefix i on weekday t is the total weight of its orders placed on that weekday during the 52 weeks that start on Monday 2017-08-07. Each demand point is thus a pickup point that receives one year of its zip area’s orders within one week. The day-to-day variation is real, and the only scale parameter is the number of folded weeks.

Warehouse and hubs. The warehouse is the seller zip prefix with the most sellers: prefix 14940 in Ibitinga, with 49 sellers. The hubs are the SP cities with the largest delivered weight, each placed at its prefix with the most orders. In this order they are São Paulo, Campinas, Guarulhos, Santos and São Bernardo do Campo, and an instance with k hubs uses the first k . On instance `large_k5_n200_T7_s1` the road distance from the warehouse to a hub lies between 253.1 and 421.6 km, so one truck trip costs between 2,467.55 and 3,810.10 BRL. The hub prefix itself is not a demand point, so its own demand is not served.

Demand points. For k hubs, the point pool holds every prefix within a straight-line radius of its nearest hub, excluding the hub prefixes, ranked by the number of orders; the pool keeps the busiest prefixes up to a fixed size (Table 1). An instance draws its n points from the pool with a fixed seed. Capacities and initial stocks follow four rules. C1: the capacity U_i of a point is the larger of its busiest folded day and a fixed number of average days. C2: its initial stock is a fixed number of average days. C3: a hub holds a fixed multiple of the capacities of its points. C4: the initial stock of a hub is a fixed number of average days of its region. Four build checks make every method able to find a feasible solution: a hub fits in one truck (A1), the points of a hub fit into m vans under the greedy packing (A2), a point capacity fits in one van (A3), and every hub has a point (A4).

Road distances. Distances come from the table service of the OSRM demo server (Luxen and Vetter, 2011), which returns the distances of the fastest routes on OpenStreetMap data. One directed matrix over the warehouse, the five hubs and the union of all point pools (420 locations) was fetched on 2026-09-24 in 81 requests of 50×50 locations; no location lacked a route. The distance between two nodes is the mean of both directions. These road distances break the triangle inequality, so the MIP could shorten a route by passing a point without delivering, which the heuristics never do. Each instance therefore replaces every distance by the shortest path through its other nodes (the Floyd-Warshall algorithm).

Parameters. Table 1 lists every parameter with its value and source. The truck is a two-axle truck; its cost per km and per trip are the minimum freight prices of the Brazilian land transport agency ANTT for general cargo (Agência Nacional de Transportes Terrestres, 2026), charged on both legs of the trip. The van is a Fiat Fiorino with a payload of 650 kg; we chose this small urban van because with a larger van the van capacity never bound on any instance. No official van cost table was found, so the van reuses the truck rate per km; this is an assumption. The holding cost is the capital cost of the goods only: the total item price divided by the total item weight of the kept orders in the fold window, times the Selic policy rate of the Brazilian central bank (Banco Central do Brasil, 2026), divided by 365. It is the same at hubs and points because the goods are the same, and it is small (0.0205 BRL per kg and day, rounded). The order data are from 2017 and 2018 while costs and rates are from 2026; no inflation adjustment is made.

Instance sets. Three sets are built (Table 2). The small set has 1 or 2 hubs, 6 to 10 points and $T = 3$ days, sized so that Gurobi proves optimality. The large set has all 5 hubs, 200 points and $T = 7$, in line with the 200-customer scale of IRP benchmarks (Archetti et al., 2017). The

Table 1: Parameters with value and source. Values are read from `config/params.toml`; the holding cost is computed from the goods value and the Selic rate and rounded here to four decimals. ANTT: Agência Nacional de Transportes Terrestres (2026); BCB: Banco Central do Brasil (2026). The rules C1 to C4 and A2 are described in Section 4.

Symbol	Meaning	Value	Source
Q^1	truck capacity (kg)	7,480	VW Delivery 11.180, payload plus body
c^1	truck cost per km (BRL)	3.9826	ANTT table, CCD, 2 axles
f^1	truck cost per trip (BRL)	451.84	ANTT table, CC
Q^2	van capacity (kg)	650	Fiat Fiorino, maker page
m	vans per hub	4	assumption (rule A2)
c^2	van cost per km (BRL)	3.9826	assumption: ANTT CCD reused
	goods value (BRL/kg)	54.302	Olist item prices
	Selic rate per year	0.1375	BCB series 432
$\eta^H = \eta^P$	holding (BRL/kg/day)	0.0205	goods value $\times$ Selic / 365
	point capacity (average days)	2.0	assumption (rule C1)
	point initial stock (days)	1.0	assumption (rule C2)
	hub capacity factor	1.5	assumption (rule C3)
	hub initial stock (days)	1.0	assumption (rule C4)
	pool radius around a hub (km)	50	assumption
	pool size (prefixes)	300	assumption
	weeks folded onto one week	52	assumption
	ALNS segment length	200	Coelho et al. (2012b)
r	ALNS reaction factor	0.7	Coelho et al. (2012b)
	scores best, better, accepted	10, 5, 2	Coelho et al. (2012b)
w	start worse share	0.05	Ropke and Pisinger (2006)
	final temperature ratio	0.001	assumption
	removal share, caps	0.1, 2, 30	assumption

Table 2: The 19 instances. k hubs, n points, T days; points per hub (smallest to largest); total demand over the horizon (kg); busiest hub-day, the largest one-day demand of one hub region (kg).

Instance	k	n	T	Points per hub	Total demand	Busiest hub-day
small_k1_n6_T3_s1	1	6	3	6	68.8	44.7
small_k1_n6_T3_s2	1	6	3	6	100.5	47.6
small_k1_n8_T3_s1	1	8	3	8	107.2	49.6
small_k1_n8_T3_s2	1	8	3	8	138.5	68.0
small_k1_n10_T3_s1	1	10	3	10	111.3	50.6
small_k1_n10_T3_s2	1	10	3	10	155.1	70.4
small_k2_n6_T3_s1	2	6	3	1 to 5	87.4	41.9
small_k2_n6_T3_s2	2	6	3	3 to 3	217.0	126.8
small_k2_n8_T3_s1	2	8	3	1 to 7	116.9	41.9
small_k2_n8_T3_s2	2	8	3	4 to 4	331.8	133.2
small_k2_n10_T3_s1	2	10	3	1 to 9	159.0	67.7
small_k2_n10_T3_s2	2	10	3	5 to 5	396.4	141.9
medium_k3_n40_T7_s1	3	40	7	7 to 20	2,131.4	221.2
medium_k3_n40_T7_s2	3	40	7	9 to 22	2,111.6	333.4
medium_k3_n80_T7_s1	3	80	7	15 to 36	3,966.5	404.7
medium_k3_n80_T7_s2	3	80	7	16 to 43	3,695.1	444.1
large_k5_n200_T7_s1	5	200	7	12 to 69	9,941.0	901.1
large_k5_n200_T7_s2	5	200	7	17 to 64	10,388.0	780.5
large_k5_n200_T7_s3	5	200	7	19 to 64	10,129.3	829.5

medium set lies in between. Figure 1 shows the network of one large instance. The last column of Table 2 is the busiest hub-day: the largest demand of one hub region on one day. It is at most 444.1 kg on the medium set and between 780.5 and 901.1 kg on the large set, against a van capacity of 650 kg. On the small and medium sets one day of demand of a hub region thus fits in one van. A route can still exceed the van capacity there if its visits bring several days of demand.

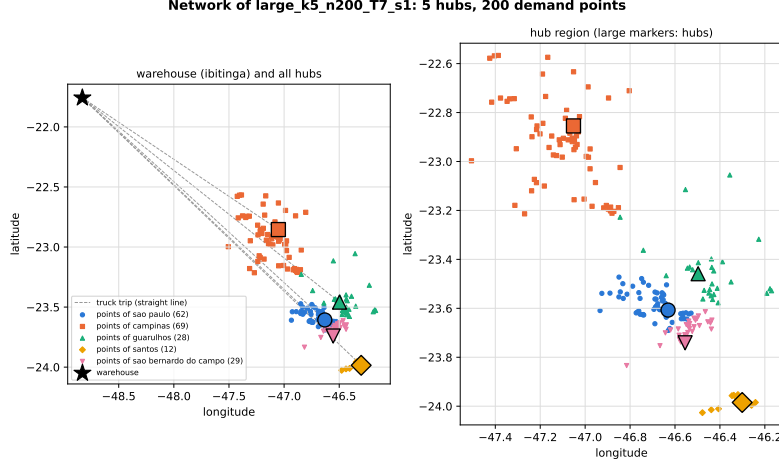


Figure 1: Network of instance `large_k5_n200_T7_s1`. Left: the warehouse in Ibitinga and the straight lines of the truck trips to the five hubs. Right: the hub region enlarged, with each point drawn in the color and marker of its hub. Lines are straight, not road geometry.

5 Methods

In the ALNS and the matheuristic a *plan* fixes the truck trips and the van routes, and the quantities follow from the plan by a rule. The greedy rule works the other way round: it sets the quantities first and builds the routes and truck trips from them.

5.1 Greedy baseline

The greedy rule processes the days in order. G1: a point receives a delivery only if its stock at the start of the day is below the demand of the day; it is then filled up, but never above what it still needs until the end of the horizon, $q_{it} = \min(U_i, \sum_{t' \geq t} d_{it'}) - I_{i,t-1}$. G2: for each hub and day, the points with a delivery are routed by nearest neighbor; a van goes to the nearest unrouted point whose delivery fits in its remaining capacity, and when no point fits, the next van starts. G3: a hub is refilled by the same rule as G1, applied to its outflow, the sum of its points' deliveries. G4: a truck drives to every hub with a positive shipment. The capacity rules C1 and C3 make G1 and G3 always feasible, and check A2 bounds the number of vans.

5.2 Adaptive large neighborhood search

Representation and quantities. The search state is a plan: a Boolean truck trip for every hub and day, and a list of routes for every hub and day. Quantities are not part of the state. They are computed after every change by the just-in-time rule J1. For one point with visit days $v_1 < \dots < v_r$ and running stock s , a visit on day v_j delivers

$$q = \max \left(0, \min \left(\sum_{t=v_j}^{v_{j+1}-1} d_{it} - s, U_i - s \right) \right), \quad (14)$$

the demand up to the day before the next visit (or to the last day), limited by the capacity. The same rule sets the hub shipments, with the truck trip days as visit days and the hub outflow as demand. J1 reports the first stockout day of a point or hub. The quantity step returns no answer when any point or hub runs out or any route carries more than Q^2 . J1 delivers as late as possible. When J1 is feasible and $\eta^P \geq \eta^H$, it therefore gives the lowest holding cost of all quantities that fit the plan: each partial sum of shipments is as small as the plan allows. The search starts from an empty plan, to which the greedy repair operator below adds every needed visit and truck trip.

Destroy operators. Each iteration draws the removal count ρ uniformly from $1, \dots, \rho_{\max}$, where $\rho_{\max}$ is a share of the current number of visits, clipped to fixed bounds (Table 1). Six destroy operators change the plan. *Random removal* removes ρ visits chosen uniformly. *Worst*

removal ranks all visits by the km saved when they are removed, and removes ρ visits drawn from the top 2ρ . *Related removal* picks one visit and removes the ρ visits of the same hub, on any day, whose points are closest to its point (Shaw, 1998); the point’s own visits on other days come first. *Sequence removal* removes up to ρ consecutive stops from one random route (Voigt, 2025). *Hub-day removal* removes all visits of one random hub on one day. *Truck trip removal* removes one random truck trip; if the previous trip of that hub cannot cover the extra days, the repair adds a later trip.

Repair. Every repair operator first applies rule R-points and ends with rule R-hubs. R-points computes J1 for all points; each point with a stockout needs one new visit on a day between its last visit before the stockout and the stockout day. An option is a day in this window together with an existing route of the hub (at its cheapest insertion position) or a new route if fewer than m routes run that day. Its cost is c^2 times the added km, and it is allowed only if the route load after insertion, with the J1 quantity of the new visit, stays within Q^2 . The *greedy repair* inserts the cheapest option over all needy points and repeats. The *regret repair* inserts first the point with the largest difference between its best and second-best option (Ropke and Pisinger, 2006). The *extra-visits repair* runs the greedy repair and then adds one visit, on a random day without a visit, for up to ρ of the points the destroy operator touched. The *extra-trip repair* runs the greedy repair and then adds one truck trip on a random day for one touched hub; it is the only way a truck trip moves earlier. R-hubs then adds a truck trip on the first stockout day of every hub until no hub runs out. If a needy point has no allowed option, the repair fails and the candidate is rejected.

One iteration. A destroy and a repair operator are drawn by roulette wheel with probability proportional to their weights (Ropke and Pisinger, 2006). After destroy and repair, the quantity step computes (q, y) ; if it gives no answer, the candidate is rejected. Visits and trips with zero quantity are then removed, and every changed route is improved by 2-opt with first improvement (Croes, 1958). A candidate with cost f' replaces the current solution with cost f if $f' < f$, or else with probability $\exp(-(f' - f)/\tau)$. The start temperature τ_0 is set so that a solution a share w worse than the initial solution is accepted with probability one half, $\tau_0 = wf_0/\ln 2$ (Ropke and Pisinger, 2006). The temperature falls geometrically with the share p of the time budget used, $\tau = \tau_0\epsilon^p$, where ϵ is the final temperature ratio. Tying the cooling to the time used lets one schedule serve fast small instances and slow large ones.

Adaptive weights. Both operators of an iteration receive the same score: one value for a new best solution, a smaller one for a solution better than the current one, and a still smaller one for a worse solution that is accepted. After every segment of iterations, the weight of operator j becomes $w_j(1 - r) + r\pi_j/\theta_j$ (Ropke and Pisinger, 2006), where π_j is its score and θ_j its number of uses in the segment; an unused operator keeps its weight. Segment length, reaction factor r and scores are those of Coelho et al. (2012b) (Table 1).

5.3 Matheuristic

The matheuristic is the same ALNS with the same operators, parameters and seeds; only the quantity step differs (rule M1). It first applies J1. Only when J1 gives no answer, Gurobi solves a linear program (LP) over the quantities of the fixed plan:

$$\min \sum_{t \in \mathcal{T}} \left(\eta^H \sum_{h \in H} I_{ht} + \eta^P \sum_{i \in N} I_{it} \right) \quad (15)$$

subject to (3) to (6) with one quantity q_{it} per point and day in place of $\sum_b q_{ibt}$, the bounds $0 \leq q_{it} \leq \min(U_i, Q^2)$ if i is visited on day t and $q_{it} = 0$ otherwise, $0 \leq y_{ht} \leq \min(C_h, Q^1)$ if hub h has a trip on day t and $y_{ht} = 0$ otherwise, and $\sum_{i \in r} q_{it} \leq Q^2$ for every route r of day t . Routing cost is fixed by the plan, so the LP minimizes holding cost only. The LP is built once per run; each call changes the bounds and replaces the route rows. If the LP is infeasible, the

candidate is rejected. Since J1 already has the lowest holding cost whenever it is feasible and $\eta^P \geq \eta^H$, which holds in every run here (Section 5.2), the LP can only help when J1 overloads a van: it can move part of a delivery to an earlier visit of the same point. The run counts the LP calls and the calls that return quantities (LP rescues).

5.4 Solution checker

One checker, separate from all methods, verifies every final solution against the rules of Section 3 (shipments only on trips, capacities, ML rules, stock balances, one visit per point and day, at most m routes, deliveries only on visits, no stockout) with a tolerance of 10^{-4} kg, and recomputes the cost. Each run passes the checker before its solution is saved, and the analysis script checks all 466 saved solutions again before it writes any table.

6 Experimental setup

All runs used one computer with 8 logical CPUs, 4 runs in parallel, Python 3.12.13 and Gurobi 13.0.3 with 1 thread per Gurobi call. Gurobi solves the MIP on the small set with a limit of 600 s. The greedy rule is deterministic and runs once per instance. The ALNS and the matheuristic run with each of 10 seeds per instance, with time limits of 10 s (small), 60 s (medium) and 120 s (large). The sensitivity study runs the greedy rule and the ALNS on `medium_k3_n80_T7_s1` with the holding costs η^H and η^P multiplied by 1, 10, 100, 1000 and 10000, with the medium time limit.

Runs stop on wall-clock time and the temperature depends on the time used, so a run is not bit-reproducible on another machine: two runs with the same seed diverge once their iteration speeds differ. For the small set the gap of a run is $100(f - f^*)/f^*$, with f^* the proven optimum, and the van gap compares the van costs in the same way. For the medium and large sets the reference is the best cost of any run of any method on that instance.

The heuristics are compared on the medium and large sets by the two-sided Wilcoxon signed-rank test (Wilcoxon, 1945) at level 0.05, without correction for multiple tests. The ALNS and the matheuristic are paired by seed. Since the greedy result is a single number, each heuristic is compared with it by a one-sample test on the 10 differences to the greedy cost. Costs are rounded to 10^{-6} BRL before the test. With 10 nonzero differences of the same sign the exact two-sided p-value is $2 \cdot 2^{-10} = 0.0020$, the smallest value the test can give with 10 pairs.

7 Results

7.1 Small instances

Gurobi proves optimality on 12 of the 12 small instances (Table 3). The longest proof takes 111.6 s and all 12 take 195.6 s together. The ALNS and the matheuristic reach the proven optimum in every run on every small instance. The greedy gap lies between 0.00 % and 4.87 %. The van gap of the greedy rule is larger, up to 20.66 %, because the truck trips are a large, fixed share of every small optimum and the greedy rule uses the same number of trips as the optimum on these instances.

7.2 Medium and large instances

Table 4 summarizes the costs on the medium and large sets and Figure 2 shows the gap of every run. The mean gap of the greedy rule to the best known cost lies between 5.07 % and 19.18 %. Both heuristics beat the greedy cost in every run on every instance, and each of these comparisons gives $p = 0.0020$ (Table 5). The largest gap of any matheuristic run is 0.63 %, and that of any ALNS run is 6.16 %. The ALNS has a few poor runs: on `medium_k3_n80_T7_s2` and `large_k5_n200_T7_s2` its standard deviation is 780.21 and 1,052.65 BRL, against 18.72 and 93.95 BRL for the matheuristic. On the medium set the matheuristic never called the LP, so this difference comes from diverging search paths, not from the LP.

Table 3: Small instances. Optimum and solution time of Gurobi; gap to the optimum (%) of the greedy rule, and its van gap (%); best and mean gap (%) of the ALNS and the matheuristic (MH) over 10 seeds. Numbers are rounded for display from `results/tables/*.csv`.

Instance	Optimum (BRL)	Gurobi (s)	Greedy gap	Greedy van gap	ALNS best/mean	MH best/mean
small_k1_n6_T3_s1	4,205.49	1.1	4.69	20.66	0.00 / 0.00	0.00 / 0.00
small_k1_n6_T3_s2	4,901.82	0.4	0.00	0.00	0.00 / 0.00	0.00 / 0.00
small_k1_n8_T3_s1	4,577.86	19.9	2.28	7.78	0.00 / 0.00	0.00 / 0.00
small_k1_n8_T3_s2	5,019.58	2.4	4.34	12.32	0.00 / 0.00	0.00 / 0.00
small_k1_n10_T3_s1	4,584.28	111.6	2.79	9.61	0.00 / 0.00	0.00 / 0.00
small_k1_n10_T3_s2	5,054.29	8.1	4.87	13.62	0.00 / 0.00	0.00 / 0.00
small_k2_n6_T3_s1	3,754.12	0.5	1.21	3.53	0.00 / 0.00	0.00 / 0.00
small_k2_n6_T3_s2	7,673.57	0.2	0.03	0.00	0.00 / 0.00	0.00 / 0.00
small_k2_n8_T3_s1	7,155.43	4.0	1.66	8.30	0.00 / 0.00	0.00 / 0.00
small_k2_n8_T3_s2	8,883.27	0.3	0.02	0.00	0.00 / 0.00	0.00 / 0.00
small_k2_n10_T3_s1	7,279.78	46.2	1.24	5.77	0.00 / 0.00	0.00 / 0.00
small_k2_n10_T3_s2	8,865.16	0.9	0.11	0.31	0.00 / 0.00	0.00 / 0.00

Table 4: Medium and large instances: best, mean and standard deviation of the total cost (BRL) over 10 seeds; mean and largest gap to the best known cost (%); mean number of iterations; and mean number of LP rescues per run of the matheuristic (MH). The greedy rule runs once. Numbers are rounded for display.

Instance	Method	Best	Mean	Std	Mean gap	Max gap	Iterations	LP rescues
medium_k3_n40_T7_s1	Greedy	32,630.77			5.34	5.34		
	ALNS	30,975.81	30,978.31	1.75	0.01	0.02	11,866	
	MH	30,977.01	30,978.79	1.71	0.01	0.02	10,277	0.0
medium_k3_n40_T7_s2	Greedy	30,285.20			5.07	5.07		
	ALNS	28,823.58	28,841.95	13.22	0.06	0.11	12,250	
	MH	28,824.90	28,838.22	13.30	0.05	0.11	9,946	0.0
medium_k3_n80_T7_s1	Greedy	40,239.56			9.31	9.31		
	ALNS	36,812.43	36,840.24	19.52	0.08	0.19	5,594	
	MH	36,810.99	36,835.88	13.27	0.07	0.11	4,677	0.0
medium_k3_n80_T7_s2	Greedy	39,583.83			19.18	19.18		
	ALNS	33,214.56	33,718.20	780.21	1.52	5.59	5,059	
	MH	33,212.31	33,242.21	18.72	0.09	0.21	4,610	0.0
large_k5_n200_T7_s1	Greedy	66,695.79			11.26	11.26		
	ALNS	59,983.04	60,104.81	81.52	0.27	0.46	5,213	
	MH	59,944.16	60,046.88	65.74	0.17	0.36	4,576	220.8
large_k5_n200_T7_s2	Greedy	60,569.49			9.24	9.24		
	ALNS	55,445.33	55,866.19	1,052.65	0.76	6.16	5,210	
	MH	55,497.00	55,579.86	93.95	0.24	0.63	4,133	201.2
large_k5_n200_T7_s3	Greedy	67,542.31			9.31	9.31		
	ALNS	61,836.47	61,908.47	38.83	0.19	0.29	5,167	
	MH	61,790.10	61,888.00	61.76	0.16	0.31	5,262	180.5

The difference between the ALNS and the matheuristic is not significant on any of the seven instances; the p-values lie between 0.1934 and 0.6953. On the three large instances the matheuristic has the lower mean cost, but the sign of the median paired difference changes between instances.

7.3 When the LP matters

The matheuristic made 0 LP calls on the small and medium sets. On the large set it made 6,032 LP calls in total, of which 6,025 returned feasible quantities. The mean number of LP rescues per run lies between 180.5 and 220.8 per instance, and between 25 and 628 over single runs. J1 overloads a route when a destroy step removes a later visit of a point, so that an earlier visit must carry more. Once the matheuristic accepts a rescued plan, that plan keeps a route that J1

Table 5: Two-sided Wilcoxon signed-rank tests on the medium and large instances: median cost difference (BRL) and p-value for ALNS minus greedy, matheuristic (MH) minus greedy, and ALNS minus MH. A negative median means that the first method is cheaper.

Instance	ALNS – Greedy		MH – Greedy		ALNS – MH	
	Median	p	Median	p	Median	p
medium_k3_n40_T7_s1	−1,652.78	0.0020	−1,652.58	0.0020	−0.53	0.1953
medium_k3_n40_T7_s2	−1,436.14	0.0020	−1,450.56	0.0020	3.37	0.3008
medium_k3_n80_T7_s1	−3,399.17	0.0020	−3,400.73	0.0020	2.97	0.6953
medium_k3_n80_T7_s2	−6,341.17	0.0020	−6,341.60	0.0020	15.56	0.2324
large_k5_n200_T7_s1	−6,579.68	0.0020	−6,623.37	0.0020	89.69	0.1934
large_k5_n200_T7_s2	−5,037.82	0.0020	−5,030.56	0.0020	−20.95	0.6250
large_k5_n200_T7_s3	−5,634.36	0.0020	−5,652.51	0.0020	6.83	0.5566

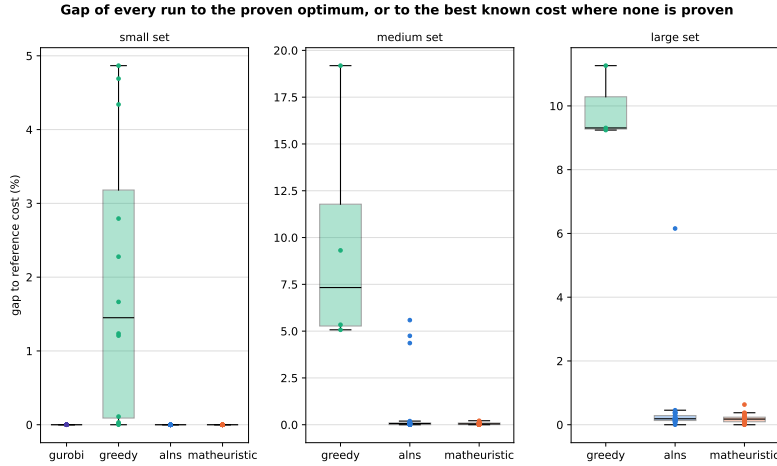


Figure 2: Gap of every run to the proven optimum (small set) or to the best known cost of the instance (medium and large sets). Each dot is one run; boxes show the quartiles.

overloads, and every candidate made from it calls the LP again until that route changes. The ALNS rejects every candidate that J1 overloads. Counting rejections for any reason, it rejected 39.8 candidates per large run on average, against 0.23 for the matheuristic. The rescues gave no significant cost difference within the time limit, and the matheuristic ran fewer iterations than the ALNS on two of the three large instances (Table 4).

7.4 Holding cost sensitivity

At the real capital cost, holding is 0.27 % of the mean ALNS cost on the sensitivity instance, and the cost consists almost only of truck trips and van routes (Table 6, Figure 3). Up to a multiplier of 100 the mean number of ALNS truck trips stays at 6.0, and the mean number of visits rises only from 261.7 to 270.7. At multipliers 1000 and 10000 the ALNS switches to more frequent and smaller deliveries: the mean number of truck trips rises to 13.0 and 17.7, and the number of visits from 261.7 to 353.8 and 431.1. The greedy rule does not react to the holding cost at all, so its plan stays the same and its holding cost grows in proportion to the multiplier. The ALNS is cheaper than the greedy rule at every multiplier, with $p = 0.0020$ in each case.

8 Conclusion

On 19 instances built from real Olist orders and road distances, the MIP proves optimality on the small instances, and the ALNS and the matheuristic find every proven optimum in every run. On the medium and large instances both beat the greedy rule in every run. The LP of the matheuristic acts only when J1 overloads a van, which in these runs happened only on the large instances.

Table 6: Holding cost sensitivity on `medium_k3_n80_T7_s1`: mean total cost and its parts (BRL), mean number of truck trips and of visits, per holding multiplier. ALNS means are over 10 seeds; the greedy rule runs once. Numbers are rounded for display.

Multiplier	Method	Total	Truck	Van	Holding	Trips	Visits
1	ALNS	36,848.41	18,044.18	18,705.66	98.58	6.0	261.7
1	Greedy	40,239.56	18,044.18	22,060.00	135.38	6.0	259.0
10	ALNS	37,683.75	18,044.18	18,695.00	944.57	6.0	262.1
10	Greedy	41,457.98	18,044.18	22,060.00	1,353.81	6.0	259.0
100	ALNS	45,786.27	18,044.18	19,059.23	8,682.86	6.0	270.7
100	Greedy	53,642.25	18,044.18	22,060.00	13,538.08	6.0	259.0
1000	ALNS	85,979.29	38,148.93	22,044.38	25,785.98	13.0	353.8
1000	Greedy	175,484.94	18,044.18	22,060.00	135,380.76	6.0	259.0
10000	ALNS	216,300.19	53,138.97	26,077.00	137,084.23	17.7	431.1
10000	Greedy	1,393,911.78	18,044.18	22,060.00	1,353,807.61	6.0	259.0

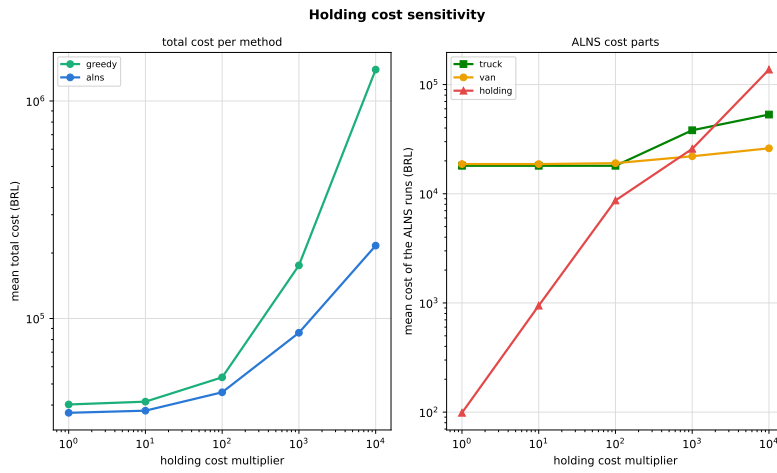


Figure 3: Holding cost sensitivity on `medium_k3_n80_T7_s1`. Left: mean total cost of the greedy rule and the ALNS. Right: mean truck, van and holding cost of the ALNS runs. Both axes are logarithmic.

The holding cost covers only the capital tied up in stock, so it is small, and routing decides almost everything at the real value. The first echelon uses direct trips, although the Guarulhos and São Bernardo do Campo hubs lie close to the São Paulo hub, so one truck tour could serve several hubs. The demand is a year of orders folded onto one week and is known in advance. The distances come from the public OSRM demo server on one date and are fastest-route distances. The van cost per km reuses the truck rate. The matheuristic showed no significant gain, and the operators were not tested one by one, although Voigt (2025) advises against relying on adaptive selection alone.

Future work follows from these limits: truck tours over several hubs in the first echelon, a holding cost that includes storage and handling, demand that is uncertain when the plan is made, a switch-off study (ablation) of each operator, and calling the LP on more plans, for example on every new best plan when the holding cost is high.

References

- Yossiri Adulyasak, Jean-François Cordeau, and Raf Jans. Formulations and branch-and-cut algorithms for multivehicle production and inventory routing problems. *INFORMS Journal on Computing*, 26(1):103-120, 2014. doi: 10.1287/ijoc.2013.0550.
- Agência Nacional de Transportes Terrestres. Resolução ANTT n° 6.084, de 16 de julho de 2026. <https://anttlegis.antt.gov.br/>, 2026. Anexo II, Tabela A, carga geral.
- Claudia Archetti, Luca Bertazzi, Gilbert Laporte, and Maria Grazia Speranza. A branch-and-cut algorithm for a vendor-managed inventory-routing problem. *Transportation Science*, 41(3):382-391, 2007. doi: 10.1287/trsc.1060.0188.

- Claudia Archetti, Luca Bertazzi, Alain Hertz, and M. Grazia Speranza. A hybrid heuristic for an inventory routing problem. *INFORMS Journal on Computing*, 24(1):101-116, 2012. doi: 10.1287/ijoc.1100.0439.
- Claudia Archetti, Natasha Boland, and M. Grazia Speranza. A matheuristic for the multivehicle inventory routing problem. *INFORMS Journal on Computing*, 29(3):377-387, 2017. doi: 10.1287/ijoc.2016.0737.
- Banco Central do Brasil. Taxa de juros - meta Selic definida pelo Copom, SGS série 432. <https://api.bcb.gov.br/dados/serie/bcdata.sgs.432/dados?formato=json>, 2026. Value 13.75 percent per year on 2026-09-24.
- Sara Charaf, Duygu Taş, Simme Douwe P. Flapper, and Tom Van Woensel. A branch-and-price algorithm for the two-echelon inventory-routing problem. *Computers & Industrial Engineering*, 196:110463, 2024a. doi: 10.1016/j.cie.2024.110463.
- Sara Charaf, Duygu Taş, Simme Douwe P. Flapper, and Tom Van Woensel. A matheuristic for the two-echelon inventory-routing problem. *Computers & Operations Research*, 171:106778, 2024b. doi: 10.1016/j.cor.2024.106778.
- Leandro C. Coelho and Gilbert Laporte. The exact solution of several classes of inventory-routing problems. *Computers & Operations Research*, 40(2):558-565, 2013. doi: 10.1016/j.cor.2012.08.012.
- Leandro C. Coelho, Jean-François Cordeau, and Gilbert Laporte. Consistency in multi-vehicle inventory-routing. *Transportation Research Part C: Emerging Technologies*, 24:270-287, 2012a. doi: 10.1016/j.trc.2012.03.007.
- Leandro C. Coelho, Jean-François Cordeau, and Gilbert Laporte. The inventory-routing problem with transshipment. *Computers & Operations Research*, 39(11):2537-2548, 2012b. doi: 10.1016/j.cor.2011.12.020.
- Leandro C. Coelho, Jean-François Cordeau, and Gilbert Laporte. Thirty years of inventory routing. *Transportation Science*, 48(1):1-19, 2014. doi: 10.1287/trsc.2013.0472.
- G. A. Croes. A method for solving traveling-salesman problems. *Operations Research*, 6(6):791-812, 1958. doi: 10.1287/opre.6.6.791.
- R. Cuda, G. Guastaroba, and M. G. Speranza. A survey on two-echelon routing problems. *Computers & Operations Research*, 55:185-199, 2015. doi: 10.1016/j.cor.2014.06.008.
- Katyanne Farias, Khaled Hadj-Hamou, and Claude Yugma. Model and exact solution for a two-echelon inventory routing problem. *International Journal of Production Research*, 59(10):3109-3132, 2021. doi: 10.1080/00207543.2020.1746428.
- Thiago A. Guimarães, Leandro C. Coelho, Cleder M. Schenekemberg, and Cassius T. Scarpin. The two-echelon multi-depot inventory-routing problem. *Computers & Operations Research*, 101:220-233, 2019. doi: 10.1016/j.cor.2018.07.024.
- Dennis Luxen and Christian Vetter. Real-time routing with OpenStreetMap data. In *Proceedings of the 19th ACM SIGSPATIAL International Conference on Advances in Geographic Information Systems*, pages 513-516. ACM, 2011. doi: 10.1145/2093973.2094062.
- C. E. Miller, A. W. Tucker, and R. A. Zemlin. Integer programming formulation of traveling salesman problems. *Journal of the ACM*, 7(4):326-329, 1960. doi: 10.1145/321043.321046.
- Olist. Brazilian e-commerce public dataset by Olist. Kaggle dataset olistbr/brazilian-ecommerce, version 2, 2021. URL <https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce>. License CC BY-NC-SA 4.0. Accessed 2026-09-24.
- David Pisinger and Stefan Ropke. A general heuristic for vehicle routing problems. *Computers & Operations Research*, 34(8):2403-2435, 2007. doi: 10.1016/j.cor.2005.09.012.
- S. U. K. Rohmer, G. D. H. Claassen, and Gilbert Laporte. A two-echelon inventory routing problem for perishable products. *Computers & Operations Research*, 107:156-172, 2019. doi: 10.1016/j.cor.2019.03.015.
- Stefan Ropke and David Pisinger. An adaptive large neighborhood search heuristic for the pickup and delivery problem with time windows. *Transportation Science*, 40(4):455-472, 2006. doi: 10.1287/trsc.1050.0135.
- Paul Shaw. Using constraint programming and local search methods to solve vehicle routing problems. In *Principles and Practice of Constraint Programming: CP98*, volume 1520 of *Lecture Notes in Computer Science*, pages 417-431. Springer, 1998. doi: 10.1007/3-540-49481-2_30.
- Oğuz Solyalı and Haldun Süral. A branch-and-cut algorithm using a strong formulation and an a priori tour-based heuristic for an inventory-routing problem. *Transportation Science*, 45(3):335-345, 2011. doi: 10.1287/trsc.1100.0354.
- Oğuz Solyalı and Haldun Süral. An effective matheuristic for the multivehicle inventory routing problem. *Transportation Science*, 56(4):1044-1057, 2022. doi: 10.1287/trsc.2021.1123.
- Stefan Voigt. A review and ranking of operators in adaptive large neighborhood search for vehicle routing problems. *European Journal of Operational Research*, 322(2):357-375, 2025. doi: 10.1016/j.ejor.2024.05.033.
- Frank Wilcoxon. Individual comparisons by ranking methods. *Biometrics Bulletin*, 1(6):80, 1945. doi: 10.2307/3001968.